

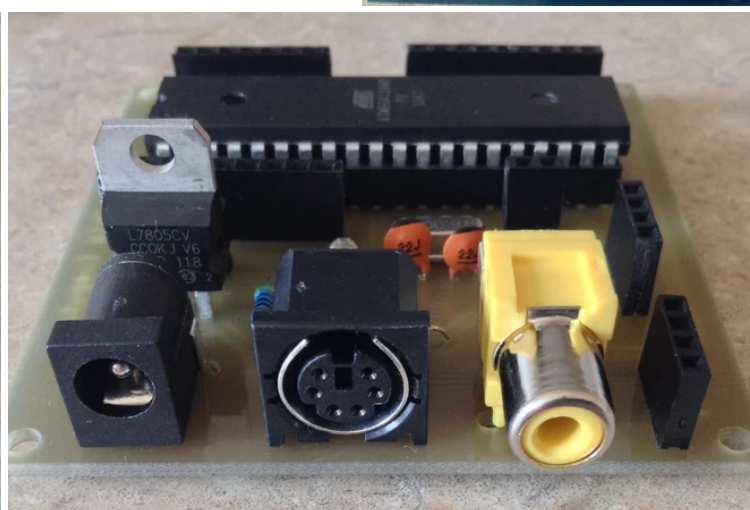
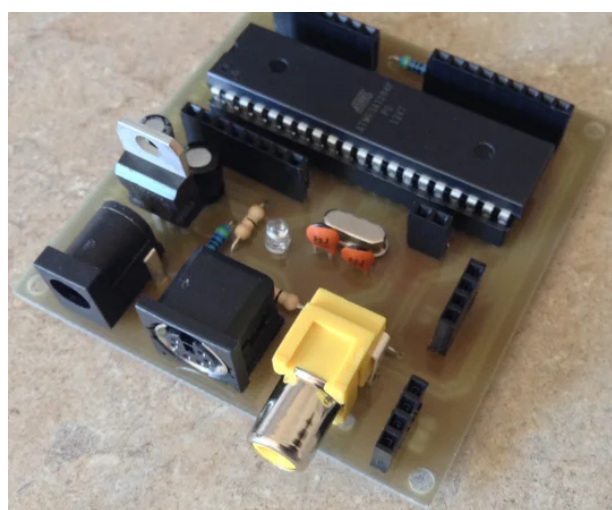
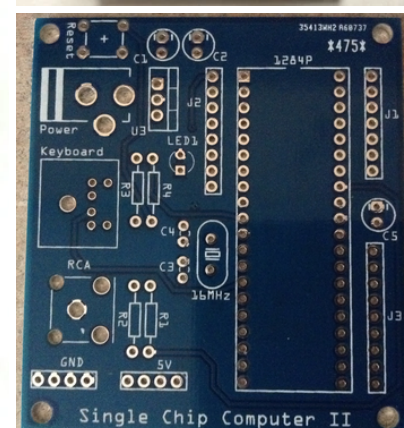
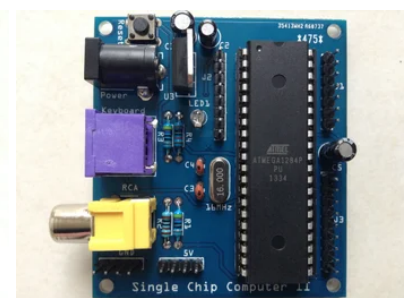
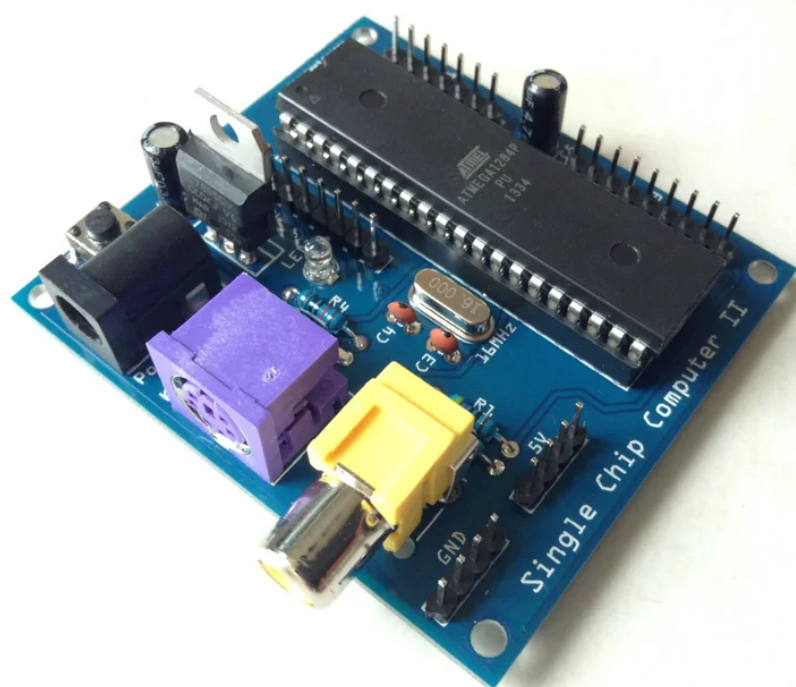
AUTODESK
Instructables

Single Chip Computer: Easy to Produce AVR BASIC Computer

By [dan14](#) in [CircuitsMicrocontrollers](#)



Introduction: Single Chip Computer: Easy to Produce AVR BASIC Computer



This instructable will document and explain my latest project, a standalone computer system based upon a single chip (IC); the ATmega 1284P. The 1284P is responsible for all aspects of the system, including running the BASIC interpreter, generation RCA video signals and reading keyboard input. This computer system runs TinyBASIC just like my Arduino BASIC Shield but this project is completely standalone meaning no Arduino is needed and only a single AVR is

needed.

This single chip computer has a video output via RCA, allows a PS/2 keyboard to be connected, allows circuits and components to be connected using the IO headers and runs the TinyBASIC programming interpreter. All components are placed on a single sided PCB which has a simple layout and is easy to produce. The onboard ATmega 1284P is running on a 16MHz crystal. This computer is also low power (it does not draw large amounts of current as it uses a micro-controller which are generally low power devices and the circuit current draw is limited by the voltage regulator to 1A) and some of the PCB components are reusable (such as the ATmega 1284P and the resistors etc).

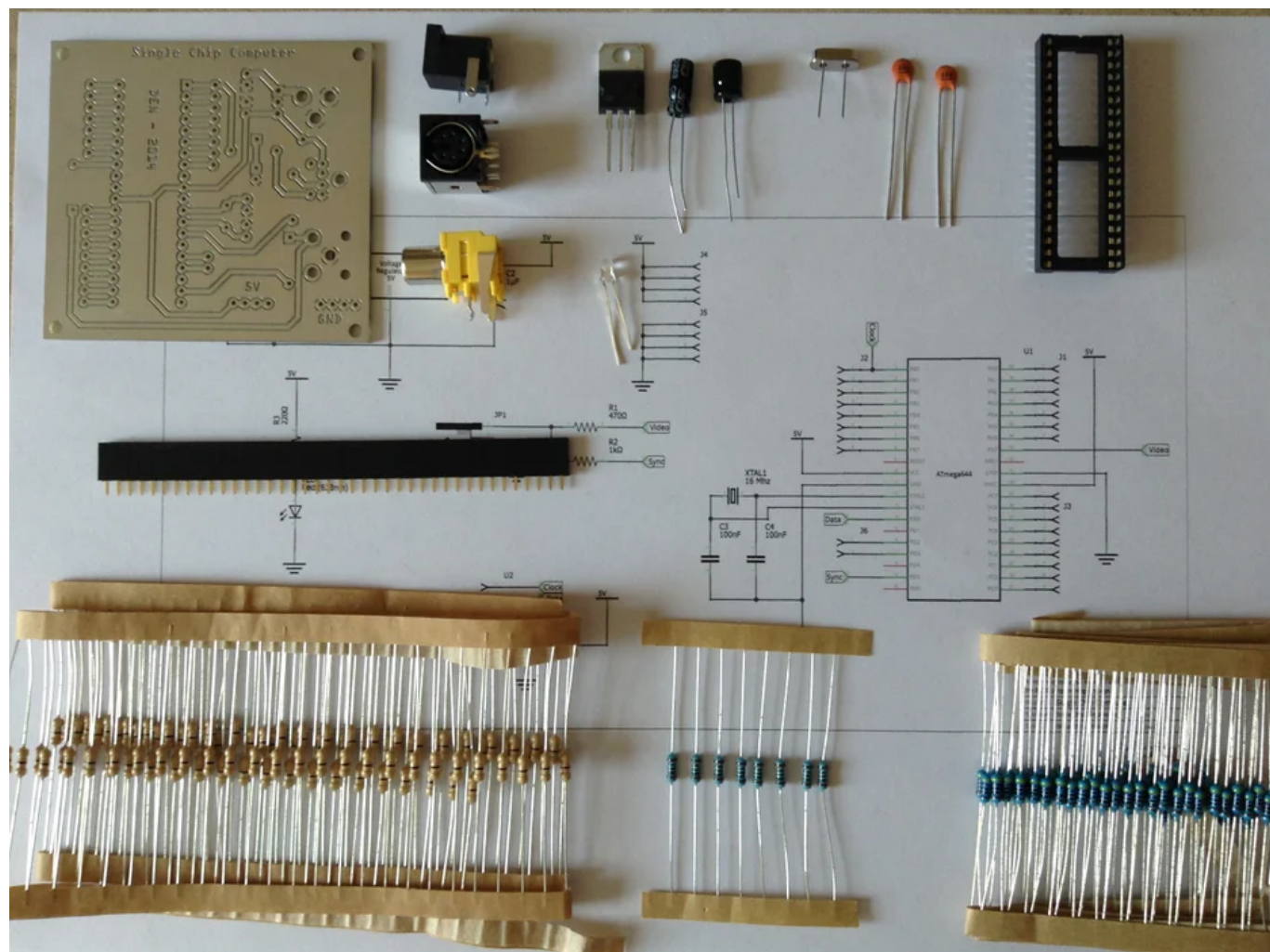
The images of the blue PCB are for the second version of this computer (details for this board can be found on Hackaday projects: <https://hackaday.io/project/1260-single-chip-avr-basic-computer>

Update (07/12/14): I updated the design of the computer by adding an additional EEPROM IC allowing full size BASIC programs to be saved. The new design of this computer is documented on Hackaday projects at the following link: <https://hackaday.io/project/3537-avr-basic-computer-v01>

I had PCBs for this new computer design created professionally at a board house and I am selling the remaining PCBs on eBay at the following link: <http://www.ebay.co.uk/itm/121510974344>

Update (28/07/14): I have ordered more PCBs (sent PCB files to the board house) and will be putting the boards up on eBay when I receive them (I will post the listing link when I receive the boards). I have slightly updated the design of the PCB to include a USBasp programming header, a few more GPIO pins and a header which allows a small PCB containing a 64Kbit EEPROM IC (an EEPROM storage card) to be connected (used for storing BASIC programs).

Step 1: Required Components



The component list for this computer is fairly low as only a single IC is required:

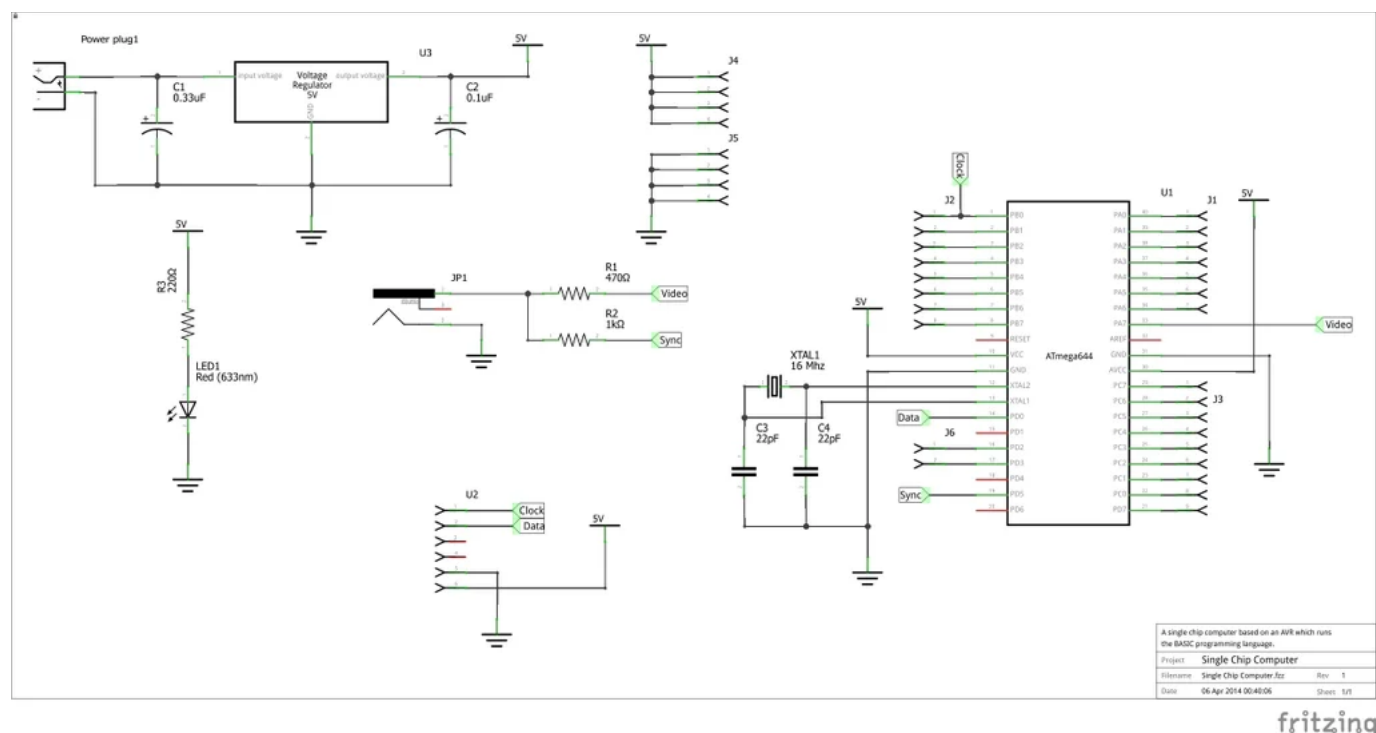
- 1 x ATmega 1284P Microcontroller
- 1 x DIL 40 Pin 0.6" IC Holder
- 1 x 40 Pin Female Pin Header (to be cut down into smaller headers)
- 1 x 16 MHz Crystal
- 2 x 22pF Ceramic Capacitors
- 1 x 2.1mm Barrel Jack
- 1 x L7805 Voltage Regulator
- 1 x 0.1uF Electrolytic Capacitor
- 1 x 0.33uF Electrolytic Capacitor
- 1 x RCA PCB Mount Socket
- 1 x PS/2 Mini Din PCB Mount Socket
- 2 x 470 Ohm Resistors
- 1 x 1k Ohm Resistor
- 2 x 0 Ohm resistors (used as jumpers)
- 1 x 3mm LED
- 1 x PCB / Strip-board / Matrix-board (depending on how the computer is going to be constructed. If a PCB is not used, different components may be needed as some have footprints such as the RCA socket which cannot be placed on strip-board)

Additionally, tools are needed to construct the PCB including a soldering iron, solder, needle nose pliers and wire cutters.

To operate (use) the computer, a PS/2 keyboard, RCA enabled display (such as a TV) and a

power source (such as a PP3 battery or wall mounted PSU) are needed. To bootload and program the 1284P, a programmer is needed (I used an Arduino UNO as an ISP).

Step 2: The Circuit



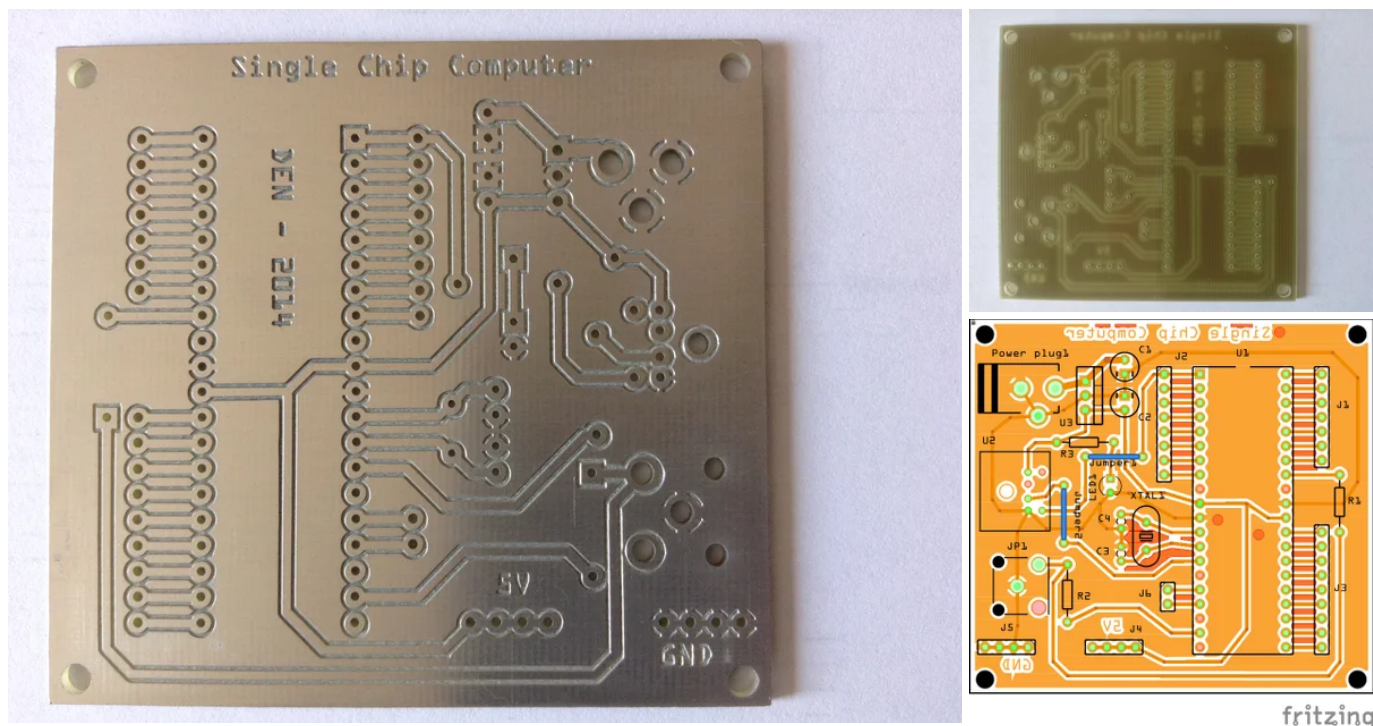
I designed the circuit diagram in the program Fritzing as it had all the required footprints readily available. The circuit is fairly simple as the ATmega 1284P has the minimum hardware connections required (along with a 16MHz crystal) to operate along with the TVout connections (RCA socket) and the PS/2 keyboard connections (mini din socket)

The TVout pins are PD5 (for sync) and PA7 (for video) and the PS/2 keyboard pins are PD0 (for data) and PB0 (for clock). To power the circuit, a 5V voltage regulator is used (along with two capacitors) to allow various input voltages from different source to be connected (such as a 9V PP3 battery or 12V wall mounted PSU). A 2.1mm barrel jack (same power jack as Arduinos) is used to allow power supplies to be connected. An on-board LED is used to indicate when a power supply is connected.

Some female pin headers were used to allow connections to the input/output (IO) pins of the ATmega 1284P, connections to the second UART (serial port) of the 1284P and connections to 5V and ground from. This allows various circuits and components to be connected to the system such as LEDs, potentiometers, LDRs etc and allows power to be taken from the board. The serial port header is present on the PCB but serial communications have not been enabled within the TinyBASIC Plus source code as serial communications clash with the keyboard library I used.

The Fritzing project file can be found attached (as a RAR file) along with PDF and JPEG images of the PCB and the schematic diagram. Note, although the schematic diagram shows an ATmega 644, the 644 and the 1284P are pin compatible and the 644 is the one in the Fritzing library so it was used instead of creating a new part; this circuit uses a 1284P but it may be possible to use a 644 if needed (the TinyBASIC code would need to be altered to change the RAM usage as it is currently too high for the 644).

Step 3: The PCB



Once I had used the program Fritzing to design the PCB, it was sent off to a PCB manufacture to be produced using the isolation milling process. The PCB is single sided as a double sided PCB was not needed, due to the fairly low component count on the PCB. Only two jumper parts are needed; I used two 0 ohm resistors.

Fritzing was used as it had all the parts I needed readily available. The Fritzing parts library had the right footprints for the RCA socket and the PS/2 mini din socket meaning they did not need to be created. Other programs such as KiCAD and Eagle could be used if needed but, in this case it was easier to use Fritzing.

Instead of creating a PCB, a piece of strip/matrix board could be used (or even a breadboard) but other connectors would be needed due to some of the footprints; the RCA socket and the 2.1mm barrel jack are two examples of components which cannot be attached to stripboard.

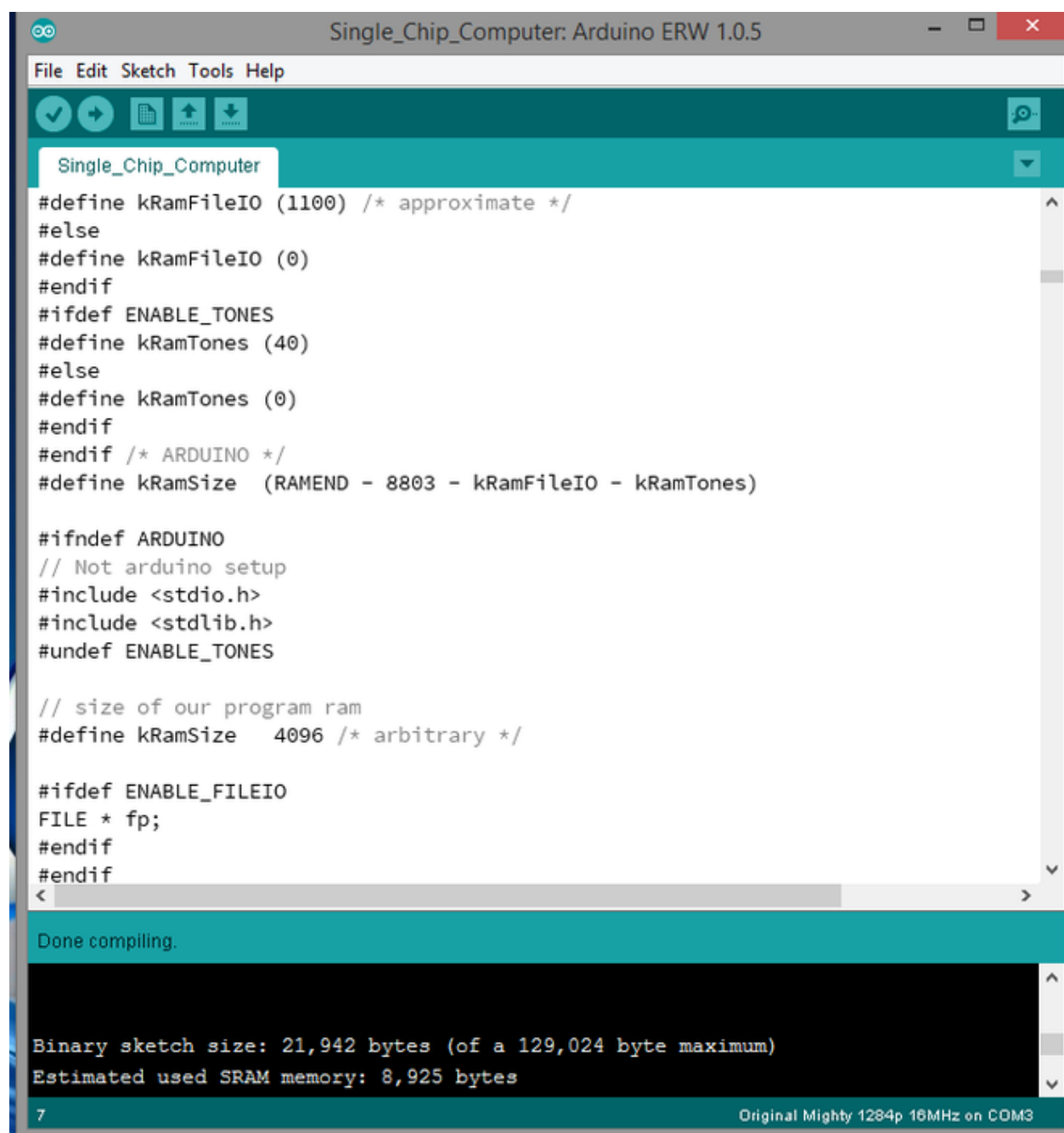
The PCB could be optimised in certain ways such as making it smaller by moving the components (such as the female pin headers around the ATmega 1284P) closer together and if a double sided PCB was being used (with features such as through-hole plating), the jumper parts could be removed.

The Fritzing file for this project which contains the PCB layout and the circuit diagram can be found attached as a RAR archive on the circuit page. The images on this page show the PCB after it had arrived and the Fritzing PCB view after routing had been completed.

Update:

I have attached the PCB as etchable PDFs (which were generated using Fritzing) as a ZIP archive on this page.

Step 4: The Firmware



```
Single_Chip_Computer: Arduino ERW 1.0.5
File Edit Sketch Tools Help
Single_Chip_Computer
#define kRamFileIO (1100) /* approximate */
#else
#define kRamFileIO (0)
#endif
#ifdef ENABLE_TONES
#define kRamTones (40)
#else
#define kRamTones (0)
#endif
#endif /* ARDUINO */
#define kRamSize (RAMEND - 8803 - kRamFileIO - kRamTones)

#ifndef ARDUINO
// Not arduino setup
#include <stdio.h>
#include <stdlib.h>
#undef ENABLE_TONES

// size of our program ram
#define kRamSize 4096 /* arbitrary */

#ifdef ENABLE_FILEIO
FILE * fp;
#endif
#endif
< >
Done compiling.
Binary sketch size: 21,942 bytes (of a 129,024 byte maximum)
Estimated used SRAM memory: 8,925 bytes
7 Original Mighty 1284p 16MHz on COM3
```

The standard TinyBASIC Plus sketch had to be modified in order to create a standalone computer system. The unmodified version of TinyBASIC Plus uses the serial port of the AVR to allow users to interact with the software and create programs. I imported the TVout library and replaced all serial output calls (Serial.write() etc) with TVout calls (TV.print() etc). The standard PS/2 keyboard library clashes with TVout causing strange output to be shown on the TV and the keyboard not being read correctly but, after some research, I came across a library known as PS2uartKeyboard which uses the XCK0 pin and RX0 pin of an AVR to connect to the keyboard in order to allow a PS/2 keyboard to work with the TVout library. This library can be found here near the bottom of the page:

<https://code.google.com/p/arduino-tvout/issues/detail?id=38>

(Actual library link [here](#))

I tested the PS2uartKeyboard library and the TVout library together and they worked fine so I added the PS2uartKeyboard library to the TinyBASIC Plus sketch which resulted in creating a

working standalone computer system. The serial calls which read serial input in TinyBASIC Plus were replaced with keyboard calls to read the keyboard input. As the TVout library, the PS2uartKeyboard library and TinyBASIC Plus sketch were all running on the same AVR, the computer requires only a single chip (one AVR) in order to operate. I started TVout using the following line:

```
TV.begin(PAL, 720, 480);
```

which starts TVout at a higher resolution than the default in order to fit some of the text output from TinyBASIC Plus on a single line as at the default resolution, some of the text wraps to the next line. Note, a resolution of 720x480 is not being generated here as it is divided down by the TVout library to a lower resolution. Starting TVout like this uses around 8KB of the ATmega 1284Ps SRAM (for the video buffer). I had to alter the RAMEND value in TinyBASIC Plus in order to allow enough memory for the video buffer required by TVout and I also changed the break key used in TinyBASIC Plus (it was previous CTRL+C) because control keys are not supported by this PS2 keyboard library (I used the escape key instead).

I had to use the original ATmega 1284P 16MHz bootloader in order to stop any TVout glitches which occur when using the optiboot bootloader (the original bootloader does not cause these glitches). When compiling the sketch in the Arduino ERW IDE, the sketch size is less than 22KB and the estimated SRAM usage of the AVR is nearly 9KB (leaving around 8KB for the TVout video buffer). The TinyBASIC Plus sketch can be found attached on this page.

The ATmega 1284P Arduino bootloader can be found here:

<https://github.com/fakufaku/mighty-1284p>

The original TinyBASIC Plus code can be found here:

<https://github.com/BleuLlama/TinyBasicPlus>

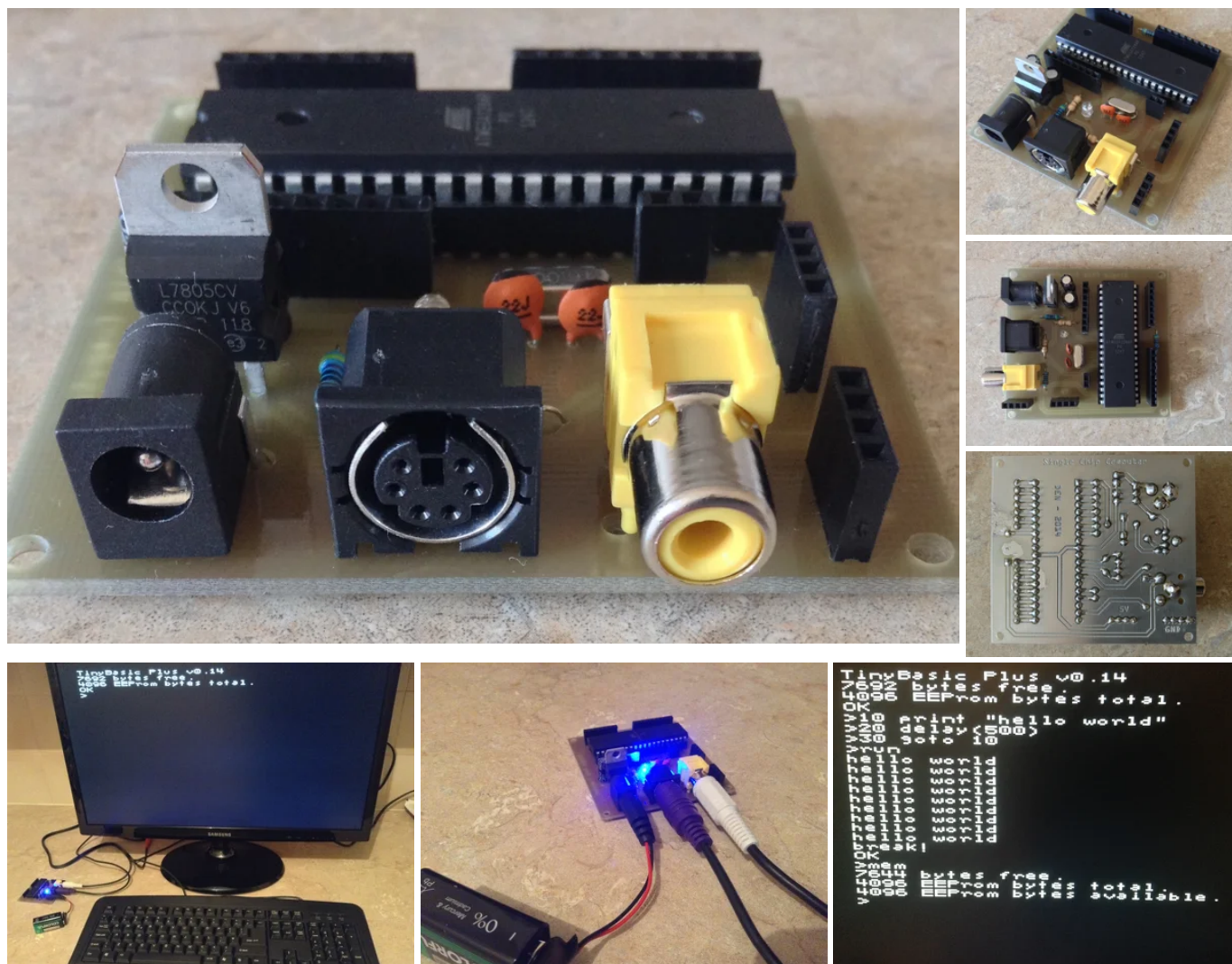
The TVout library can be found here:

<https://code.google.com/p/arduino-tvout/>

Please note, TinyBASIC Plus, Arduino software (bootloaders, IDE etc), the PS2uartKeyboard library and the TVout library all hold there own software licences which must be followed if using any of the creators code.

Update (23/06/2014): I have attached the hex file for the TinyBASIC Plus sketch (compiled using the Arduino IDE) so the Arduino IDE does not have to be used to program the ATmega 1284P. I got this file from the temporary directory created by the Arduino IDE during the compilation stage of the sketch so I am not sure if it the correct one (please let me know if anyone has any problems using it).

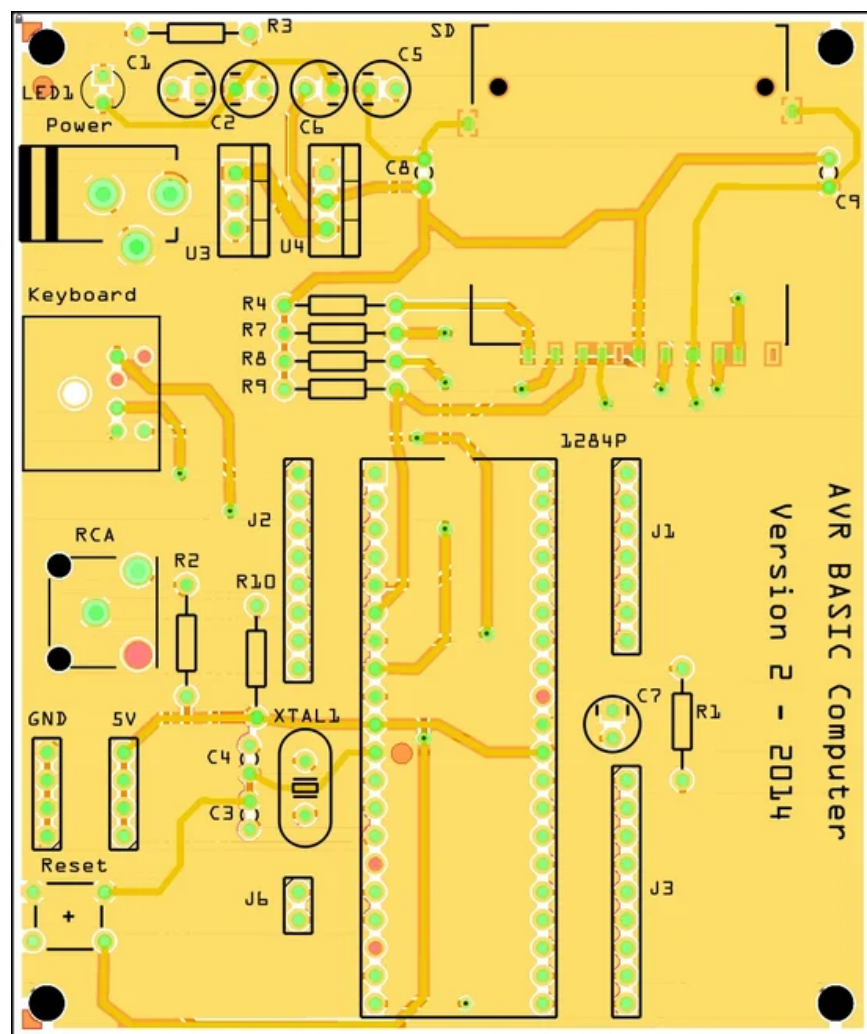
Step 5: Completed PCB



Once I had all the components and had received the PCB, I assembled it and carried out some basic multimeter tests ensuring no short circuits were present (mainly ensuring no pins had shorts to the ground plane). No issues were present with this PCB and everything appeared to operate correctly once assembled so I programmed the TinyBASIC Plus sketch from the Arduino IDE (using an Arduino UNO as an ISP).

Once I had connected a TV and a PS/2 keyboard, I was met with the TinyBASIC prompt which indicates the amount of memory available for programs, the software version etc showing everything was running OK. TinyBASIC Plus reported 7692 bytes (7.692KB) are available for programs and the ATmega 1284P has 4KB of EEPROM to save programs to. The images on this page show the completely assembled PCB as well as TinyBASIC being displayed on the connected TV.

Step 6: Future Versions



fritzing

Although TinyBASIC Plus supports SD cards for BASIC program storage, I chose not to include an SD card holder on this version of the PCB in order to keep the component count low (as a 3.3V regulator and some form of level shifting would be required as I am running the ATmega 1284P at 16MHz meaning a 5V supply is needed) and because I had issues with using the SD card library with the TVout library.

After looking into the SD card library in the Arduino IDE directory, I altered the file "Sd2PinMap.h" (in the utility folder) to include the ATmega 1284P as only the ATmega 644P is supported in there (the Arduino IDE does not know which pins of the ATmega 1284P are the SPI pins, so they need to be defined in this file); this was done by simply adding a define to the Sanguino section of the "Sd2PinMap.h" file like so:

```
#elif defined(__AVR_ATmega644P__) || defined(__AVR_ATmega644__) ||
defined(__AVR_ATmega1284P__)
// Sanguino
```

After doing this and enabling file IO in TinyBASIC Plus, programs stored on an attached SD card can be auto-run on the computer but when typing in the "FILES" command, the displayed data is garbage (it shows up as random unreadable characters); the files stored on the SD card should be displayed but they are not. After looking further into the SD card library, I came across a file called "SdFile.cpp" (in the utility folder) which has serial port calls (Serial.print, Serial.println etc) which I believe is the cause of the random characters being displayed on the attached TV (when

using the "FILES" command, serial data may be sent to the TV which does not get displayed correctly). Before I can include an SD card holder on any future version of the single chip computer, I need to test altering this file to output to an attached TV rather than over serial.

For a future version of this computer, I would use a PCB service which would allow a coloured board with a silk screen to be created. The PCB would also be double sided, removing the need for the jumper parts and possibly use some surface mount components to reduce its size. Ideally, a second version would include an SD card slot, but the TV displaying issue (mentioned above) needs to be fixed before an SD card slot could be added. Alternatively, EEPROM ICs could be used (e.g. two 24K128 ICs; only one would be used at a time for program storage which would be selected using a jumper attached to an input pin of the ATmega 1284P) for storing the BASIC programs (this has the advantage of removing the need for a second 3.3V voltage regulator and level shifting components which are needed for an SD card); using EEPROM though would mean the computer is no longer a single chip one, but a multi chip computer. A design of a future version of this computer, with an SD card slot, can be seen in one of the above images (designed in Fritzing).

I may also get this first version of the single chip computer produced using a PCB service which allows a coloured PCB, silk screen and through-hole plating which would remove the need for the jumper parts and give a more professional look to the computer system.